

Problem Reconstruction

Why a stateless or retrieval-only assistant is insufficient, and what follows from that.

System Conversational Memory Intelligence System

Grounding domain GTM AI, B2B SaaS revenue teams

Handbook v1.0, section 3

Author Dheeraj Pranav

Date 19 July 2026

Companions failure_analysis.md, first_principles.md, historical_timeline.pdf

EVIDENCE STANDARD

Claims carry one of four tags, the same scheme used across all four Deliverable 1 artifacts: **VERIFIED** (stated in a cited source or in my own production experience), **INFERENCE** (my reading of what the evidence implies, not claimed by the source), **ASSUMPTION** (a working assumption about my deployment, unmeasured), or **TO MEASURE** (a claim I refuse to assert until the Deliverable 3 baseline produces a number). **No benchmark result appears in this document.** The naive baseline is Deliverable 3 and has not been built.

1. The problem, stated without naming a solution

The handbook asks that the problem be written before the intended architecture is named. This section contains no mention of memory systems, retrieval, embeddings, or storage. It is the statement I would give to someone who had never heard of any of them.

An assistant that helps a person with an ongoing, long-running relationship cannot currently carry anything forward from one conversation to the next. It cannot use what it was told last week, it cannot tell what it was told last week from what it was told last year, it forgets corrections immediately, it has no way to distinguish something it should hold onto from something it should never have heard, and it has no way to prove that what it holds about one person was not shown to another.

Everything below is a consequence of that paragraph. The order matters: constraints and failures are derived from the problem, not selected to justify an architecture I had already chosen.

2. Setting, actors, and conditions of failure

2.1 Setting

A GTM assistant deployed inside a B2B SaaS vendor's revenue stack. It prepares call briefs, answers questions about an account's history, and drafts follow-ups. It is multi-tenant: the vendor sells it to many customer organisations whose data sits on shared infrastructure.

INFERENCE This domain is chosen because it supplies naturally adversarial conditions rather than contrived ones. Near-duplicate entities across tenants, preferences with genuine expiry dates, regulated personal data arriving incidentally, and an isolation requirement with commercial consequences all occur here by default. A general-purpose chatbot setting would require inventing these conditions. It is also the domain of my last six years of production work, which means the failure cases below are recognisable to me rather than imagined.

2.2 Who is affected, and how

ACTOR	WHAT THEY NEED TO PERSIST	WHAT FAILURE COSTS THEM
Account executive	Buyer preferences, stated objections, who the decision maker is now, what was promised on the last call	Re-reads transcripts before every call, or walks in with a stale fact and loses credibility with the buyer
SDR	Which accounts were already researched, what the outcome was, which angles were already tried	Repeats outreach the account has already declined
RevOps analyst	Why the system recommended what it did, on which evidence	Cannot audit or defend the system's behaviour to their own leadership
Buyer (data subject, not a user)	Nothing. They need <i>non</i> -persistence of what they did not consent to	Personal information retained indefinitely without a decision having been made
Vendor	That tenant A's memories are unreachable from tenant B	A single cross-tenant disclosure is an existential commercial event

The fourth row is the one most easily omitted. The buyer never uses the assistant and never sees it, yet a large share of the personal data flowing through it is theirs. A design that only considers users will not produce the right retention policy.

2.3 Conditions under which the problem bites

- **Long gaps between sessions.** Days to months. Nothing survives a session boundary unless deliberately carried.
- **Volume beyond replay.** An enterprise account accumulates years of calls and threads.
- **Facts with expiry.** Tooling, personnel, budget, and requirements all change mid-cycle.
- **Lexical collisions across tenants.** Two customers selling to companies with the same name, in near-identical language.
- **Incidental sensitive data.** Personal details arrive unbidden in sales conversation.
- **Cold start.** A brand-new account, where the correct behaviour is to retrieve nothing and say so.

3. Importance: the cost of leaving it unsolved

Three consequences, ordered by how recoverable they are. The ordering is the point: it is what determines which failures the architecture is allowed to tolerate.

RECOVERABLE · ABANDONMENT

ASSUMPTION An assistant that must be re-briefed every session costs more attention than it returns, and users stop opening it. I expect abandonment after roughly the third repeated correction. **VERIFIED** This matches what I saw on the Novartis Decision Engine, where field representatives ignored recommendations they could not understand or influence. Adoption is gated on the system visibly incorporating what the user tells it, and a tool that is not used has no accuracy worth discussing.

RECOVERABLE · WRONG ACTION TAKEN CONFIDENTLY

A stale fact served without a date is worse than no fact. In a revenue setting it produces a specific, visible error in front of a buyer: pitching an integration the account migrated off, addressing someone who left. The damage is to trust in the account relationship, not just to a metric.

UNRECOVERABLE · DISCLOSURE AND RETENTION FAILURE

Returning one tenant's memory to another, or being unable to honour an erasure request, cannot be corrected after the fact. No later improvement undoes the disclosure. These two are why the architecture treats isolation and deletion as invariants rather than features, and why the handbook makes both non-negotiable gates.

4. Constraints

ASSUMPTION Every number below is a target I am setting now so that Deliverable 3 has something to measure against and Deliverable 4 has something to design against. None is measured. All are revisable with a recorded reason.

CONSTRAINT	TARGET	WHY THIS NUMBER
Context	$\leq 2,000$ tokens of a 16,000-token working context	About 12%. Instructions, tool definitions, and the turn itself need the rest.
Latency	$p95 \leq 300$ ms for retrieve, rank, and construct	Must fit inside a roughly 3 s agent turn without being the visible bottleneck.
Cost	$\leq 15\%$ of per-turn token spend	Above this the feature is hard to justify in a per-seat product.
Data	$50 \text{ tenants} \times 200 \text{ users} \times 10^4$ memories $\approx 10^8$ total	Mid-market B2B SaaS deployment.
Privacy	Deletion ≤ 60 s typical, ≤ 24 h guaranteed and tested	A guarantee I can actually meet beats one I cannot.
Isolation	Zero cross-tenant reads, tested adversarially	Section 3 admits no tolerance here.
Correctness	Beat the naive baseline on a fixed evaluation set	Improvement is not a well-formed claim without a fixed distribution.
Time	Single engineer, part time, roughly 8 weeks	Forces the non-goals in section 7 to be real.

The first three constraints pull against correctness. Budget, latency, and cost all push toward retrieving less; correctness pushes toward retrieving more. **INFERENCE** Ranking quality is the only thing that buys correctness without spending budget, which is why it is the component most worth investing effort in, and why a design that spends its complexity elsewhere is probably mis-allocated.

5. Prior approaches and the assumption each makes

Summary only. Full assumption decomposition, worked failure traces, and the failure taxonomy are in `failure_analysis.md`.

APPROACH	LOAD-BEARING ASSUMPTION	WHERE IT BREAKS
A. Stateless prompting	Corrections are a property of the conversation, not of the world	The same correction is re-issued every session and never takes effect
B. Full conversation replay	Attention is a sufficient relevance mechanism, so no explicit selection is needed	$O(n^2)$ prompt cost against a linear conversation; recall degrades for facts in the middle of the input; stale and current facts carry equal weight
C. Rolling summarisation	Compression losses are unimportant and summariser errors do not accumulate	Single-mention high-stakes facts are dropped preferentially and irreversibly; each round re-ingests the last, so hedged claims harden into asserted ones
D. Retrieval-only (single-signal)	Similarity to the query is an adequate proxy for what belongs in context	Similarity ranks wording, not validity, so a superseded fact outranks the current one; chunks invert out of context; no admission, deletion, or isolation path
E. Long context as substitute	A window is a place to keep things	A window is per-request working memory with no persistence, write path, deletion semantics, tenant boundary, or audit trail, at any size

THE ONE THAT MATTERS MOST

Approach E is the common objection to building any of this, and it is the one worth answering precisely. **VERIFIED** Liu et al. (arXiv:2307.03172) show that usable recall from long contexts is position-dependent and worst in the middle, so effective capacity is smaller than nominal capacity. **INFERENCE** But the deeper error is categorical, not quantitative. Growing the window changes the *size* of the selection budget and nothing else about the problem's structure. It does not touch persistence, lifecycle, isolation, or observability, because a context window has none of those properties at any size.

6. Failure evidence

Two representative traces. The full set of eleven, with the assumption each violates and whether it is recoverable, is in `failure_analysis.md`.

6.1 Similarity ranks wording, not validity

Query: "What CRM does Acme use?"

RETRIEVED CHUNK	DATE	COSINE	TRUTH
"We're a Salesforce shop, everything routes through SFDC."	2025-02-11	0.91	superseded
"We finished the HubSpot migration last quarter."	2026-04-30	0.84	current

INFERENCE The superseded fact ranks higher because it is phrased more directly on-topic. This is not an embedding-quality problem. The ordering is correct with respect to the objective the embedder was trained on. The fix cannot live inside the similarity signal and has to be an explicit validity interval plus a supersession relation between memories. **TO MEASURE** The exact cosine scores above are illustrative; the real distribution, and how often the superseded fact actually outranks the current one, is a number the Deliverable 3 baseline produces on the fixed set.

6.2 A chunk is an utterance, and utterances invert

Chunk retrieved: "I don't think we need SS0 for the pilot."

Actual exchange: AE: Should we scope SS0 into the pilot?
Buyer: I don't think we need SS0 for the pilot.
AE: Security will probably push back.
Buyer: Fair. Put it in. Non-negotiable for prod.

Injected alone, the chunk asserts the opposite of the account's actual position. **INFERENCE** The unit of storage has to be an extracted, typed, resolved claim rather than a span of raw text. This argument is independent of every ranking concern above, which is why it forces a separate extraction stage rather than a better ranker.

7. Derived requirements

`first_principles.md` derives ten capabilities forwards from eight premises about the setting, then checks that result against the failure-driven list. Both directions are shown there. The capabilities, in short form:

ID	CAPABILITY	FORCED BY
C1	Durable cross-session write	The model is a pure function of its context, so nothing persists unless something writes it
C2	Admission policy, not blanket capture	Conversation is low-precision and high-redundancy; some information is ineligible for retention
C3	Typed representation with provenance, confidence, validity	Utterances invert out of context; facts have validity intervals; deletion needs provenance
C4	Source of truth separate from a rebuildable index	Deletion must be authoritative somewhere; evaluation requires re-projection without data loss
C5	Multi-signal ranking with conflict resolution	Similarity cannot express validity; relevance alone over-selects duplicates
C6	Budgeted context construction, including ordering	Context is finite, monotonically costly, and position-sensitive
C7	Lifecycle: update, consolidate with citation, decay, expire, delete	The world revises facts; retention rights are revocable; redundancy accumulates
C8	Isolation as an invariant, not a filter	Tenants are mutually distrusting and share infrastructure
C9	Decision observability	Stage decisions are not visible in the output, so failures cannot be localised without traces
C10	Reproducible offline evaluation against the baseline	Retrieval quality is distributional, not per-query

A FINDING ABOUT METHOD

C9 and C10 do not appear anywhere in the failure-driven derivation. They arrive only from the forwards derivation. **INFERENCE** Breaking four prior designs surfaces the failures those designs can *exhibit*, never the failures I would be unable to *detect*. A capability whose absence causes silent undetectability is invisible to a purely failure-driven method. That is the strongest argument for having done the derivation in both directions, and it is the result I would most want challenged in review.

7.1 Non-goals

Recorded now so that Deliverable 4 does not quietly expand into them.

- **Not a general knowledge base.** Only information observed in conversation. Product documentation is a separate corpus with different freshness and ownership properties.
- **Not learned retrieval.** Ranking signals stay hand-specified and inspectable in v1. A learned ranker needs labelled data that does not exist yet and would make C9 much harder.
- **Not cross-user shared memory.** Team-level memory raises consent and access-control questions the current premises do not answer. Deferred with the reason recorded.
- **Not model-agnostic in v1.** One provider, one embedding model, behind a swappable interface. Reproducibility requires pinning.

8. Open questions

These shape Deliverable 4 and are the questions I most want attacked in the design review.

1. **Does typed extraction trade one failure for a worse one?** A wrongly extracted fact looks authoritative in a way a wrongly retrieved chunk does not. *Validation:* run both over the same transcripts in Deliverable 3 and compare the *character* of the errors, not only the rate.
 2. **Should supersession be inferred automatically, or should conflicting memories be injected with their dates and resolution left to the model?** Automatic supersession is an irreversible write based on a fallible judgement.
 3. **What deletion consistency window can I actually hold under load?** A defensible number beats an undeliverable guarantee.
 4. **Does write-time importance scoring earn its cost?** Importance is often only apparent in hindsight. **VERIFIED** Generative Agents scores it at write time; **INFERENCE** that setting has no per-seat cost pressure.
 5. **How large must a labelled evaluation set be before its numbers mean anything?** Without an answer, C10 is ceremony rather than measurement.
-

Sources

1. Lewis, P., Perez, E., Piktus, A., et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2020. arXiv:2005.11401
2. Park, J. S., O'Brien, J. C., Cai, C. J., et al. *Generative Agents: Interactive Simulacra of Human Behavior*. 2023. arXiv:2304.03442
3. Packer, C., Wooders, S., Lin, K., et al. *MemGPT: Towards LLMs as Operating Systems*. 2023. arXiv:2310.08560
4. Vaswani, A., Shazeer, N., Parmar, N., et al. *Attention Is All You Need*. 2017. arXiv:1706.03762
5. Liu, N. F., Lin, K., Hewitt, J., et al. *Lost in the Middle: How Language Models Use Long Contexts*. 2023. arXiv:2307.03172. Not in handbook Appendix G; added because it supplies the direct evidence against Approach E.

AI assistance disclosure. Claude was used to draft and pressure-test prose in this document. The domain grounding, the choice of failure cases, the assumption decomposition, the constraint targets, and every verified / inference / assumption boundary are mine and are what I will defend in review. No benchmark number appears here because none has been measured.